

PHP

YAHIA Siwar

Qu'est-ce que PHP ?

- **PHP = Hypertext Preprocessor**
- Langage de script côté serveur
- Utilisé pour générer des pages web dynamiques
- Il est exécuté sur le serveur (et non sur le navigateur)

Fonctionnement de php

- Le navigateur envoie une requête
- Le serveur exécute le code PHP
- Le serveur envoie du HTML au navigateur
- ? Résultat : l'utilisateur ne voit que le HTML

PHP vs HTML / JavaScript

Langage	Type	Exécution
HTML	Statique	Navigateur
JavaScript	Dynamique	Navigateur
PHP	Dynamique	Serveur

Rôle de PHP dans le web

- Interaction avec les bases de données (MySQL, PostgreSQL...)
- Gestion des formulaires et sessions
- Génération de contenu dynamique (blogs, forums, e-commerce)
- Intégration facile avec HTML, CSS et JavaScript

Installation et configuration

Serveur local nécessaire : Apache ou Nginx

- **XAMPP** (Windows, Linux, Mac)
- **WAMP** (Windows)
- **MAMP** (Mac)

Étapes :

- Télécharger et installer le package
- Démarrer Apache et MySQL
- Placer les fichiers PHP dans le dossier htdocs ou www

Syntaxe de base

- Un script PHP commence par :

```
<?php  
// code  
?>
```

- Instruction d'affichage :

```
echo "Texte";
```

- Les commentaires :

```
// commentaire  
/* commentaire */
```

Premier script PHP

- Créer un fichier index.php
- Exemple minimal :

```
<?php  
echo "Hello World";  
?>
```

Exécution : placer le fichier dans htdocs, puis accéder via <http://localhost/index.php>

Résultat : affichage du texte **Hello World** dans le navigateur

Variables

- Une variable commence par \$:

```
<?php
$x = 42;      // int
$prix = 19.99; // float
$titre = "PHP 8"; // string
$marque = true; // bool
$fruits = ["pomme", "banane"]; // array
?>
```

- Types de données

```
String (texte)
Integer (entier)
Float (réel)
Boolean (true/false)
Array (tableau)
```

Les opérateurs

```
Arithmétiques : + - * /
Comparaison : == != > <
Logiques : && ||
```

Les Structures conditionnelles

Permettent de prendre des décisions dans un programme

1) Structure if:

```
if (condition) {
    // code
}
```

Exemple

```
if ($age > 18) {
    echo "Majeur";
}
```

2) Structure if...else

```
if (condition) {
    // si vrai
} else {
    // sinon
}
```

Exemple

```
$note = 8;
if ($note >= 10) {
    echo "Admis";
} else {
    echo "Refusé";
}
```

Les Structures conditionnelles

3) Structure if...elseif...else

```
if (condition1) {
    // code
} elseif
(condition2) {
    // code
} else {
    // code
}
```

```
$note = 15;
if ($note >= 16) {
    echo "Très bien";
} elseif ($note >= 10)
{
    echo "Admis";
} else {
    echo "Échec";
}
```

Les Structures conditionnelles

4) Conditions imbriquées

```
$age = 20;

if ($age >= 18) {
    if ($age >= 60) {
        echo "Senior";
    } else {
        echo "Adulte";
    }
}
```

5) Opérateur ternaire

Version simplifiée de if...else

```
(condition) ? valeur1 : valeur2;
```

Exemple

```
$age = 17;
echo ($age >= 18) ? "Majeur" : "Mineur";
```

Les Structures conditionnelles

6) Structure switch

```
switch ($variable) {
    case valeur1:
        // code
        break;
    case valeur2:
        // code
        break;
    default:
        // code
}
```

Exemple

```
$jour = "lundi";

switch ($jour) {
    case "lundi":
        echo "Début de semaine";
        break;
    case "vendredi":
        echo "Fin de semaine";
        break;
    default:
        echo "Jour normal";
}
```

Les boucles

PHP utilise 4 types de boucles :

- while
- do...while
- for
- foreach

Les boucles

1) Boucle while: Répète tant que la condition est vraie

```
while (condition) {  
    // code  
}
```

Exemple

```
$i = 1;  
while ($i <= 5) {  
    echo $i;  
    $i++;  
}
```

2) Boucle do...while: Exécute le code au moins une fois Puis teste la condition

```
do {  
    // code  
} while (condition);
```

Exemple

```
$i = 1;  
do {  
    echo $i;  
    $i++;  
} while ($i <= 5);
```

Les boucles

3) Boucle for

Utilisée quand on connaît le nombre d'itérations

```
for (initialisation; condition;
incrément) {
    // code
}
```

Exemple

```
for ($i = 1; $i <= 5; $i++) {
    echo $i;
}
```

4) Boucle foreach

```
foreach ($tableau as $valeur) {
    // code
}
```

Exemple

```
$fruits = ["pomme", "banane",
"orange"];

foreach ($fruits as $fruit) {
    echo $fruit;
}
```

Les boucles

5) foreach avec clé et valeur

```
$notes = ["Ali" => 12, "Sara" => 15];

foreach ($notes as $nom => $note) {
    echo $nom . " : " . $note;
}
```

6) Break et continue

Break permet de sortir de la boucle
continue Passe à l'itération suivante

Fonctions et modularité

```
function nomFonction() {
  // instructions
}
```

Exemple

```
function direBonjour() {
  echo "Bonjour !";
}
```

Appel d'une fonction

```
direBonjour();
```

1) Fonctions avec paramètres

```
function afficherNom($nom) {
  echo "Bonjour " . $nom;
}
```

Exemple

```
afficherNom("Ali");
```

Fonctions et modularité

2) Fonction avec valeur de retour

```
function addition($a, $b) {
  return $a + $b;
}
```

Appel d'une fonction

```
$resultat = addition(2, 3);
echo $resultat;
```

3) Portée des variables dans une fonction

1. Variable locale

Déclarée dans une fonction

Accessible uniquement dans la fonction

```
function test() {
  $x = 10;
}
```

Fonctions et modularité

2) Variable globale

Déclarée en dehors des fonctions

```
$x = 5;
function test() {
    global $x;
    echo $x;
}
```

Alternative :

```
$GLOBALS['x']
```

Fonctions intégrées (String)

```
strlen("Bonjour"); // longueur
strtoupper("php"); //convertir une chaine en majuscule
strtolower("PHP"); // convertir une chaine en minuscule
strpos("Bonjour", "j"); // position
```

Fonctions intégrées (Array)

```
count($tab); // taille  
in_array("a", $tab); // recherche  
array_push($tab, 5); // ajouter la fin  
array_unshift($tab, 5); // ajouter au début  
array_pop($tab); // supprimer dernier
```

Fonctions mathématiques

```
abs(-5); // valeur absolue  
sqrt(16); // racine carrée  
round(4.6); // arrondi  
rand(1, 10); // génère un nombre entier aléatoire  
compris entre 1 et 10 inclus.
```

Tableaux et manipulation de données

1) Tableau indexé

```
$fruits = ["pomme", "banane", "orange"];
```

Accès:

```
echo $fruits[0];
```

2) Tableau associatif

```
$personne = [
    "nom" => "Ali",
    "age" => 25
];
```

Accès:

```
echo $personne["nom"];
```

Tableaux et manipulation de données

Parcours avec for

```
for ($i = 0; $i < count($fruits); $i++) {
    echo $fruits[$i];
}
```

Parcours avec foreach

```
foreach ($fruits as $fruit) {
    echo $fruit;
}
```

Associatif :

```
foreach ($personne as $cle => $valeur) {
    echo $cle . " : " . $valeur;
}
```

Tableaux et manipulation de données

- Ajouter :

```
array_push($fruits, "kiwi"); // ajouter la fin  
array_unshift($tab, 5); // ajouter au début
```

- Fusionner :

```
array_merge($tab1, $tab2);
```

- Supprimer des éléments

```
array_pop($fruits); // supprimer dernier  
unset($fruits[1]); // index précis
```

- Fonctions utiles

```
count($tab); // taille  
in_array("a", $tab); // recherche  
sort($tab); // tri  
rsort($tab); // tri inverse  
array_unique($tab); // supprimer doublons
```

PHP et formulaire

PHP et formulaire

- Les formulaires HTML permettent aux utilisateurs de saisir des données.
- PHP récupère et traite ces données côté serveur.
- Utilisation des **superglobales** : \$_GET, \$_POST.

Applications

- Authentification

Se connecter

ou utiliser mon adresse e-mail :

Email

Mot de passe

Je n'ai pas de compte, je m'en crée un.

Se connecter

Formulaire de connexion

Application

- Recherche

Rechercher :

te

test

texte

temps

Envoyer

Application

- Envoi de messages

Votre Mail :

Sujet :

Votre message :

Code anti-spam :

Vous devez reproduire le code de l'image avant d'envoyer le message.

Structure d'un formulaire HTML

- Exemple simple :

```
<form method="POST"
action="traitement.php">
  <label>Nom :</label>
  <input type="text" name="nom">
  <input type="submit" value="Envoyer">
</form>
```

method : définit la façon dont les données sont envoyées (GET ou POST).

action : fichier PHP qui traitera les données.

name : identifiant du champ, utilisé par PHP pour récupérer la valeur.

Formulaire: method

Les méthodes principales

Il existe deux méthodes :

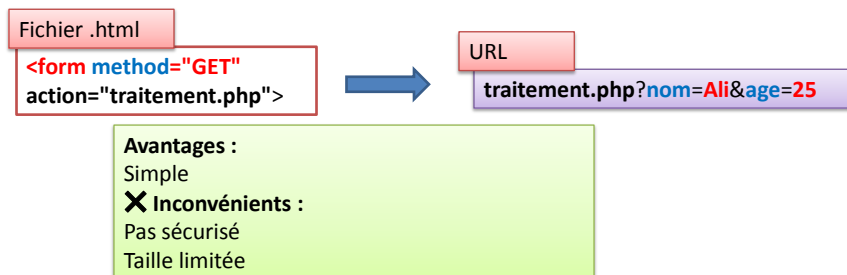
? **GET**

? **POST**

Formulaire: method GET

Caractéristiques :

- Les données sont visibles dans l'URL
- Exemple :



Formulaire: method POST

Caractéristiques :

Données envoyées dans le corps de la requête
(non visibles)
Plus sécurisé que GET

Fichier .html

```
<form method="POST"
action="traitement.php">
```

URL

données **cachées** ✗ (pas dans l'URL)

Avantages :

Pas de limite importante de taille
Utilisé pour données sensibles (mot de passe)

✗ Inconvénients :

Non visible dans l'URL alors Non bookmarkable (mettre en favori)
Impossible de Partager le lien

Récupération en PHP

• Avec **GET** :

```
$nom = $_GET['nom'];
```

Avec **POST** :

```
$nom = $_POST['nom'];
```

Bonnes pratiques

Utiliser **POST** pour : **mot de passe, données sensibles**
Utiliser **GET** pour : **recherche, filtres**

Récupération des données en PHP

- Exemple avec POST :

```
<?php
$nom = $_POST['nom'];
echo "Bonjour, " . $nom;
?>
```

`$_POST['nom']` contient la valeur saisie par l'utilisateur.
Affichage dynamique en fonction de l'entrée.

L'accès aux champs d'un formulaire

Les boutons radio

Fruits :

```
<input type="radio" name="fruit" value="Banane"> Banane
<input type="radio" name="fruit" value="Pomme"> Pomme
```

name= identifie le groupe de boutons
Value: valeur envoyer au serveur

```
$fruits = $_POST['fruit'];
```

Les cases à cocher (checkbox)

Langages :

```
<input type="checkbox" name="lang[]" value="PHP"> PHP
<input type="checkbox" name="lang[]" value="JS"> JS
```

```
foreach($_POST['lang'] as $l){
    echo $l;
}
```

L'accès au champs d'un formulaire

- Liste déroulante (select)

```
<select name="ville">
  <option value="Tunis">Tunis</option>
  <option value="Sfax">Sfax</option>
</select>
```

```
$ville = $_POST['ville'];
```

L'accès au champs d'un formulaire

- Upload fichier:

Formulaire HTML pour upload

```
<form action="upload.php" method="POST" enctype="multipart/form-data">
  <input type="file" name="fichier">
  <input type="submit" value="Envoyer">
</form>
```

type="file" → permet de choisir un fichier **enctype** → obligatoire pour envoyer des fichiers
L'attribut enctype sert à définir **comment les données sont encodées (formatées) avant d'être envoyées au serveur.**

Oui → sans multipart les données sont **modifiées (encodées)**

✓ Avec multipart → les données sont **envoyées telles quelles (surtout les fichiers)**

En PHP, on utilise la variable globale : **\$_FILES**

L'accès au champs d'un formulaire

Récupération du fichier en PHP

```
$_FILES['fichier']['name']; // nom du fichier
$_FILES['fichier']['type']; // type MIME
$_FILES['fichier']['tmp_name']; // chemin temporaire
PHP ne le met pas directement dans le dossier Il le
stocke temporairement sur le serveur Ce fichier est
supprimé automatiquement après le script .
$_FILES['fichier']['error']; // code erreur: C'est un
code qui indique si l'upload s'est bien passé. (0: aucun
problème)
$_FILES['fichier']['size']; // taille en octets
```

Structure de \$_FILES

```
$_FILES['fichier'] = [
    'name' => 'image.jpg', // nom du
    fichier
    'type' => 'image/jpeg', // type MIME
    'tmp_name' => '/tmp/xyz', // fichier
    temporaire
    'error' => 0, // code erreur
    'size' => 12345 // taille en octets
];
```

L'accès au champs d'un formulaire

- Déplacer le fichier vers un dossier:

```
move_uploaded_file($_FILES['fichier']['tmp_name'], "uploads/" .
$_FILES['fichier']['name']);
```

Exemple complet :

```
if (isset($_FILES['fichier'])) {
    $nom = $_FILES['fichier']['name'];
    $tmp = $_FILES['fichier']['tmp_name'];

    move_uploaded_file($tmp, "uploads/" . $nom);
    echo "Fichier uploadé avec succès";
}
```

Sécurisation (Fichier)

- Vérifier **les erreurs**

```
if ($_FILES['fichier']['error'] == 0) {
    echo "Pas d'erreur";
}
```

- Limiter **la taille**

```
if ($_FILES['fichier']['size'] < 1000000) { // < 1 Mo
```

- Vérifier **l'extension**

```
$extension = pathinfo($_FILES['fichier']['name'], PATHINFO_EXTENSION);
if (in_array($extension, ['jpg', 'png', 'pdf'])) {
```

Extraction de l'extension: `pathinfo($_FILES['fichier']['name'], PATHINFO_EXTENSION)`;
Exemple: `document.pdf` retourne: `pdf`

Sécurisation (Fichier)

- Créer un nouveau nom unique pour un fichier uploadé: (**éviter les conflits**):

```
$nouveauNom = uniqid() . "." . $extension;
```

- téléchargement d'un fichier

```
//On définit le chemin du fichier sur le serveur
$fichier = "uploads/test.pdf";
// Indique au navigateur :le type du fichier est un PDF
//donc il doit être traité comme un document PDF
header("Content-Type: application/pdf");
// force le téléchargement X pas affichage direct filename=test.pdf
//nom du fichier téléchargé côté utilisateur
header("Content-Disposition: attachment; filename=test.pdf");
//lit le fichier sur le serveur, envoie son contenu au navigateur et
//déclenche le téléchargement
readfile($fichier);
```

Vérification des données

Vérification des données (isset)

- **isset()**
- **Empty()**

Vérification des données (isset)

- **isset()** est une fonction PHP Elle permet de vérifier si une variable : **existe**
- et **n'est pas NULL**

Vérification des données (isset)

- Vérifier l'existence des données:

```
if (isset($_POST['nom'])) {  
    echo $_POST['nom'];  
}
```

Retourne :

✓ true → si la variable existe et ≠ NULL

✗ false → si :

la variable n'existe pas
ou elle vaut NULL

Vérification des données (empty)

- Vérifie si le champ est vide

```
if (empty($_POST['nom'])) {  
    echo "Champ vide";  
}
```

En PHP, empty() considère comme "vide" plusieurs valeurs :

- ✓ 0
- ✓ "0"
- ✓ ""
- ✓ null
- ✓ false
- ✓ tableau vide []

Sécurisation des données(htmlspecialchars())

- Éviter les attaques (XSS, injection)

```
$nom = htmlspecialchars($_POST['nom']);
```

htmlspecialchars() transforme du code HTML/JS en simple texte (chaîne de caractères)

Nettoyage et sécurité (trim())

Enlève espaces

```
$nom = htmlspecialchars(trim($_POST['nom']));
```

Nettoyage et sécurité (filter_var)

- ✓ Valider les données utilisateur
- ✓ Nettoyer les entrées (sécurité)
- ✓ Éviter les erreurs et injections
- ✓ Garantir des données correctes

Syntaxe:

```
filter_var(valeur, filtre, options);
```

valeur : donnée à traiter

filtre : type de validation/nettoyage

options : paramètres supplémentaires

Nettoyage et sécurité (filter_var)

• Validation email

```
$email = "test@gmail.com";

if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Email valide";
} else {
    echo "Email invalide";
}
```

FILTER_VALIDATE_EMAIL vérifie :

- ✓ format correct
- ✓ présence de @
- ✓ domaine valide
- ✓ extension

=> Vérifie si l'email est correct

• Validation d'une URL

```
$url = "https://example.com";

if (filter_var($url, FILTER_VALIDATE_URL)) {
    echo "URL valide";
}
```

FILTER_VALIDATE_URL vérifie :

- ✓ protocole (http, https...)
- ✓ structure correcte
- ✓ domaine valide

Nettoyage et sécurité (filter_var)

• intervalle d'un entier

```
$age = 20;

$options = [
    "options" => [
        "min_range" => 18,
        "max_range" => 30
    ]
];

if (filter_var($age, FILTER_VALIDATE_INT, $options)) {
    echo "Age valide";
}
```

- ✓ Retourne la valeur → si valide
- ✗ Retourne false → si invalide

Les sessions

Définition

- Stockage **temporaire côté serveur**. Sécurisé
 - **Utilisation:** Authentification, Panier, Données utilisateur
-
- ✓ Démarrer une session: `session_start();`
 - ✓ Stocker des données: `$_SESSION['user'] = "Ali";`
 - ✓ Lire: `echo $_SESSION['user'];`
 - ✓ Détruire: `session_destroy();`

PHP et base des données

C'est quoi une base de données

- Un système qui enregistre des informations.
- Les données sont organisées dans des tables.
- BDD est une interconnexion entre les tables

Pourquoi utiliser une base de données ?

- stocker des données (utilisateurs, produits...)
 - organiser les informations
 - faire des recherches rapides
- ☐ Exemple : site avec inscription → données stockées en base

Architecture PHP/BD

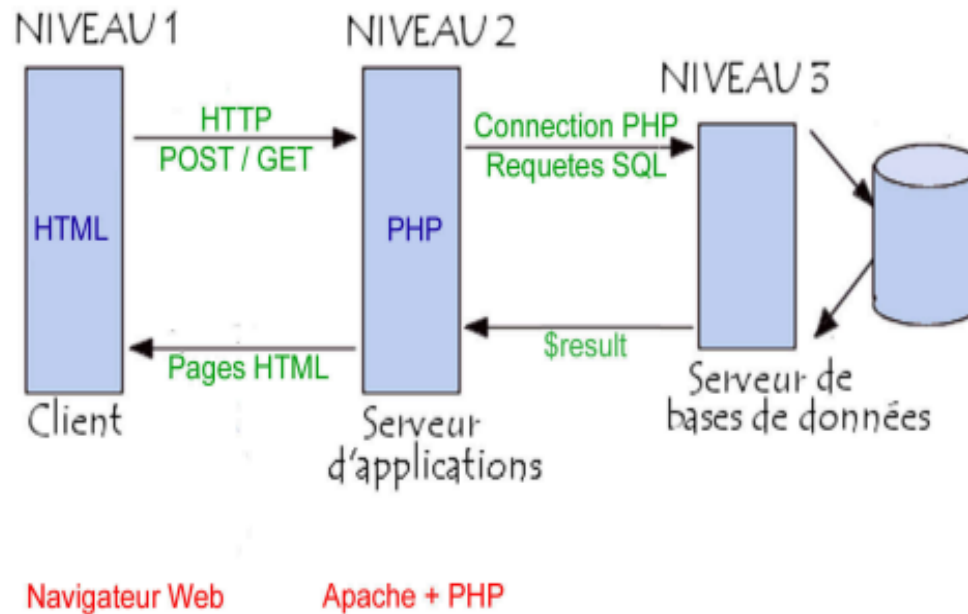


Image 1 Exemple d'architecture 3-tiers : PHP/BD

SGBD

Qui gère la base de données?

SGBD

- Système de gestion de base de données.

MySQL

Postgre
SQL

Oracle

SQLite

Microsoft SQL
server



Pour interagir avec ces systèmes d'exploitation on utilise le langage SQL

PHP et SGBD

- Comment se connecter a ces SGBD et faire le lien entre PHP et la base de données.
- Utiliser des extensions: **Mysql_**, **Mysqli_**, **PDO**
- **PDO: extension complete permet de connecter a n'importe quelle SGBD.**

PDO (PHP Data Objects)

✓ Pourquoi PDO ?

- Sécurisé ✓
- Compatible avec plusieurs SGBD ✓
- Supporte les requêtes préparées ✓

PDO

Comprends:

- Classe PDO
- La classe PDOStatement
- La classe PDOException

PDO

Connecter la BD

Sélectionner

Insérer

Supprimer

Modifier

Connexion à la base de données

```
<?php
$pdo = new PDO("SGBD:host=HOST;dbname=DBNAME;charset=utf8",
"USER", "PASSWORD");
?>
```

SGBD → type de base (mysql, pgsql, sqlite)

host → serveur (ex: localhost)

dbname → nom de la base

charset → encodage (utf8 recommandé)

USER → utilisateur (ex: root)

PASSWORD → mot de passe

Exemple

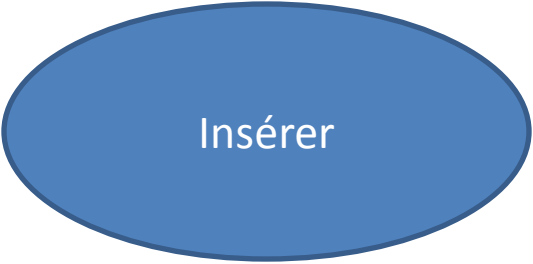
```
$pdo = new PDO("mysql:host=localhost;dbname=Personne;charset=utf8", "root", "");
```

L'**encodage (charset)** sert à définir **comment les caractères sont stockés et interprétés** dans la base de données. Sans bon encodage : é, è, à → deviennent ? ou caractères bizarres
texte arabe → illisible , emojis → non supportés

☐ Exemple :

Sans UTF-8 : Bonjour à tous → Bonjour ? tous

PDO



Insérer

Insertion dans une base de donnée

- `$sql = "INSERT INTO TABLE (col1, col2, ...) VALUES (:val1, :val2, ...)";`
`$stmt = $pdo->prepare($sql);`
`$stmt->execute([`
`':val1' => $variable1,`
`':val2' => $variable2`
`]);`

TABLE → nom de la table

col1, col2 → colonnes

:val1, :val2 → paramètres (sécurité)

\$variable1, \$variable2 → valeurs à insérer

prepare() → prépare la requête **execute()**

→ exécute

Insertion dans une base de donnée (exemple)

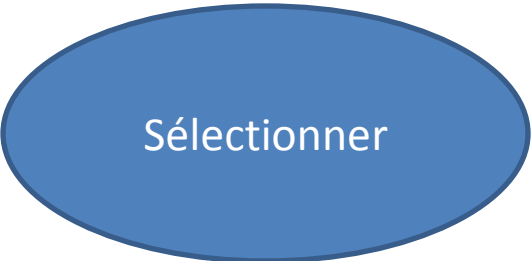
```
$nom = "Ali";
```

```
$email = "ali@mail.com";
```

```
$stmt = $pdo->prepare("INSERT INTO user(nom,  
email) VALUES (?, ?)");
```

```
$stmt->execute([$nom, $email]);
```

PDO



Sélectionner

Sélection des données

```
$sql = "SELECT col1, col2 FROM TABLE WHERE  
condition";
```

```
$stmt = $pdo->prepare($sql);
```

```
$stmt->execute();
```

```
// Récupération des résultats
```

```
$resultats = $stmt->fetchAll();
```

```
foreach ($resultats as $ligne) {  
    echo $ligne['col1'];  
}
```

SELECT col1, col2 →
colonnes à afficher **FROM**

TABLE → table **WHERE**
condition → filtre

(optionnel) **prepare()** →
prépare la requête

execute() → exécute

fetchAll() → récupère toutes
les lignes

\$ligne['col1'] → accéder à
une valeur

Sélection des données (exemple)

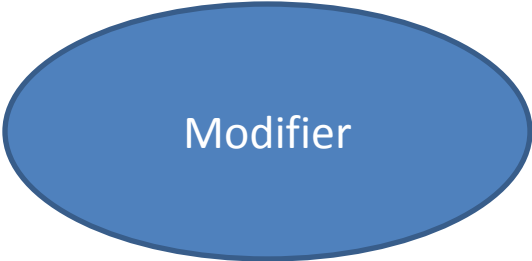
- ```
<?php
$nom = "Ali";

$stmt = $pdo->prepare("SELECT nom, email FROM
user WHERE nom = ?");
$stmt->execute([$nom]);

$resultats = $stmt->fetchAll();

foreach ($resultats as $row) {
echo $row['nom'] . " - " . $row['email'] . "
";
}
?>
```

# PDO



Modifier

# Modifier (UPDATE )

```
$sql = "UPDATE TABLE
SET col1 = :val1, col2 = :val2
WHERE condition";
```

```
$stmt = $pdo->prepare($sql);
```

```
$stmt->execute([
'val1' => $variable1,
'val2' => $variable2
]);
```

**UPDATE TABLE** → table à modifier

**SET col1 = :val1** → nouvelles valeurs

**WHERE condition** → filtre (très important !)

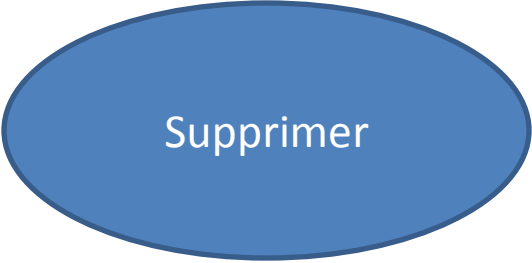
**prepare()** → sécurise la requête

**execute()** → envoie les valeurs

# Modifier (UPDATE exemple )

- `$stmt = $pdo->prepare("UPDATE user SET nom=? WHERE id=?");`
- `$stmt->execute(["Ahmed", 1]);`

# PDO



Supprimer

# Supprimer (DELETE)

- **\$sql** = "DELETE FROM **TABLE** WHERE condition";

```
$stmt = $pdo->prepare($sql);
```

```
$stmt->execute([
'val' => $variable
]);
?>
```

# Supprimer (DELETE exemple)

- `$stmt = $pdo->prepare("DELETE FROM user WHERE id=?");`  
`$stmt->execute([1]);`

# Sécurité : Requêtes préparées

## Mauvaise pratique

- `$sql = "SELECT * FROM user WHERE nom = '$nom'";`

## Bonne pratique

- `$stmt = $pdo->prepare("SELECT * FROM user WHERE nom=?");`  
`$stmt->execute([$nom]);`

☐ Protège contre **Injection SQL**